

Document number: P0739R0

Date: 2017-07-13

Reply-To:

Mike Spertus, Symantec (mike_spertus@symantec.com)

Walter E. Brown (webrown.cpp@gmail.com)

Stephan T. Lavavej (stl@exchange.microsoft.com)

Audience: {Library Evolution, Library} Working Group

Some improvements to class template argument deduction integration into the standard library

scoped_lock argument order

As Jonathan Wakely has noted, code like the following does not work:

```
mutex m;  
scoped_lock l(m, adopt_lock);
```

The point here is that when the compiler attempts to deduce `scoped_lock<m1, adopt_lock>`, there is a hard error outside the immediate context as `adopt_lock` is not lockable. Note that the same code works fine if `scoped_lock` is replaced by `lock_guard` because `lock_guard` does not have a variadic argument.

We recommend fixing this by moving the `adopt_lock_t` parameter to the front of the parameter list. This was not done originally because `scoped_lock` was originally named `lock_guard` and had to remain compatible to avoid code breakage, which is no longer a requirement now that `scoped_lock` is a different class than `lock_guard`. This is useful to fix because it will encourage programmers to migrate to consistent use of `scoped_lock` rather than choosing one or the other based on circumstances.

Wording

In the definition of class `scoped_lock` in §33.4.4.2 [thread.lock.scoped], make the following change:

```
explicit scoped_lock(MutexTypes&... m);  
explicit scoped_lock(MutexTypes&... m, adopt_lock_t);  
explicit scoped_lock(adopt_lock_t, MutexTypes&... m);  
~scoped_lock();
```

Likewise, change §33.4.4.2 [thread.lock.scoped] starting immediately before paragraph 4 as follows:

```
explicit scoped_lock(MutexTypes&... m, adopt_lock_t);  
explicit scoped_lock(adopt_lock_t, MutexTypes&... m);
```

Requires: The calling thread owns all the mutexes in `m`.

Enable variant support

The following code fails to compile

```
variant<int, double> v1(3);  
variant v2 = v1; // ill-formed!
```

As this natural code is useful and its failure is confusing, we propose that it be supported. Indeed, prior to the adoption of p0510r0 banning `variant<>`, the above code worked as expected since `variant<>` occurs in some

deduction guides in the overload set. As it is not clear that constructor template argument deduction was considered in adopting p0510r0, we would like to consider allowing `variant<>` not to produce a hard error in such cases.

Wording

Change §23.7.3.1p16 [variant.ctor] as follows

Remarks: This function shall not participate in overload resolution unless `sizeof...(Types)` is nonzero, unless `is_same_v<decay_t<T>, variant>` is false, unless `decay_t<T>` is neither a specialization of `in_place_type_t` nor a specialization of `in_place_index_t`, unless `is_constructible_v<Tj, T>` is true, and unless the expression `FUN(std::forward<T>(t))` (with *FUN* being the above-mentioned set of imaginary functions) is well formed.